NutriPredict

E2 : Developpement Backend et API

Candidat : Valentin PERIES

Certification : RNCP37827 - Developpeur en Intelligence Artificielle

Projet : NutriPredict - Prediction d'impact nutritionnel par IA

Date : Juin 2026

C6 C7 C8

E2 - Developpement Backend et API

1. Architecture technique

Le backend de NutriPredict repose sur une architecture en couches, concu pour la maintenabilite et la separation des responsabilites. Le framework choisi est **FastAPI**, un framework Python asynchrone base sur Starlette et Pydantic.

1.1 Stack technique

Composant	Technologie	Justification
Framework	FastAPI 0.115	Performance asynchrone, validation automatique, documentation OpenAPI generee
ORM	SQLAlchemy 2.0 (async)	Support du pattern async/await natif, typage fort avec Mapped[]
Base de donnees	PostgreSQL 16	Robustesse, support UUID natif, extensions JSON
Validation	Pydantic v2	Schemas de requete/reponse avec validation stricte des types
Serveur	Uvicorn (ASGI)	Serveur ASGI haute performance pour production

1.2 Structure du projet

```
backend/
    app/
        api/endpoints/    # Routeurs FastAPI (predict, products,
health_metrics)
        core/             # Configuration, securite, authentification, rate
limiter
        models/           # Modeles SQLAlchemy (User, HealthMetrics)
        schemas/          # Schemas Pydantic (requetes/reponses)
        services/         # Logique metier (MLService)
    tests/                # Tests unitaires et d'integration
    Dockerfile            # Image de production
    requirements.txt      # Dependances epinglees
```

2. Conception des endpoints API (C6)

L'API suit les principes REST et utilise le prefixe `/api/v1` pour le versionnage. Chaque endpoint est documente via les decorateurs FastAPI qui generent automatiquement la specification OpenAPI 3.1.

2.1 Endpoint de prediction (`POST /api/v1/predict`)

C'est l'endpoint principal de l'application. Il recoit trois parametres et retourne la variation de poids predite par le modele :

Parametre	Type	Contraintes	Description
user_bmr	float	500 - 5000	Metabolisme de base de l'utilisateur (kcal/jour)
biscuit_calories_100g	float	0 - 900	Apport energetique du biscuit pour 100g
portion_g	float	1 - 500	Portion consommee en grammes

La validation est assuree par Pydantic. Tout parametre hors des bornes renvoie automatiquement une erreur HTTP 422 avec un message explicite.

```
# Schema Pydantic de la requete
class PredictionRequest(BaseModel):
    user_bmr: float = Field(ge=500, le=5000, description="Metabolisme de
base")
    biscuit_calories_100g: float = Field(ge=0, le=900,
description="Calories / 100g")
    portion_g: float = Field(ge=1, le=500, description="Portion en grammes")
```

2.2 Endpoint produits (`GET /api/v1/products/{barcode}`)

Cet endpoint interroge la base interne pour recuperer les informations nutritionnelles d'un produit via son code-barres EAN-13. Il est protege par une cle d'API.

2.3 Endpoints de sante (`/api/v1/health-metrics`)

Deux endpoints permettent de gerer les mesures biometriques :

- **POST** : enregistre une nouvelle mesure. Les champs sensibles (taille, poids) sont chiffres cote serveur avant insertion.
- **GET /latest** : recupere la mesure la plus recente, dechiffree a la volee pour l'affichage.

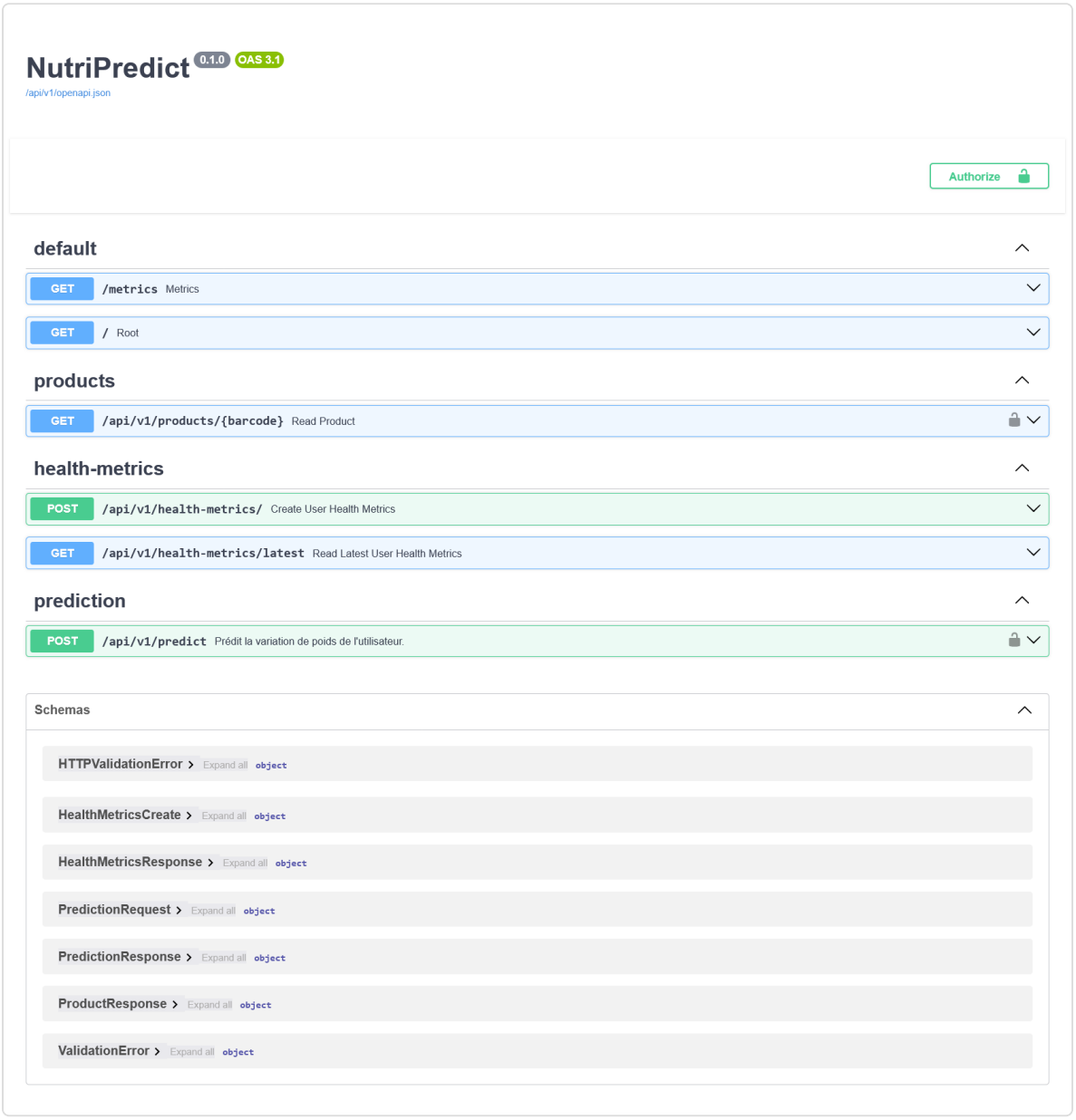


Figure 3 - Documentation interactive Swagger - Vue d'ensemble des endpoints

3. Securisation de l'API (C7)

La securite est un axe central du projet, conforme aux recommandations OWASP Top 10.

3.1 Authentification par cle API

Chaque requete aux endpoints proteges doit inclure un en-tete `X-API-KEY`. La verification est implementee comme une dependance FastAPI injectee au niveau du routeur :

```
async def verify_api_key(api_key: str = Header(..., alias="X-API-KEY")):
    if api_key != settings.API_KEY:
        raise HTTPException(status_code=403, detail="Cle API invalide")
```

3.2 Chiffrement AES-256-GCM

Les donnees biometriques sont chiffrees via AES-256-GCM (Galois/Counter Mode), un algorithme de chiffrement authentifie qui garantit a la fois la confidentialite et l'integrite des donnees.

Le processus de chiffrement :

1. La cle de 256 bits est chargee depuis la variable d'environnement `ENCRYPTION_KEY`, decodee en Base64 URL-safe.
2. Un **nonce aleatoire** de 12 octets est genere pour chaque operation (`os.urandom(12)`), garantissant qu'un meme texte clair produit un chiffre different a chaque appel.
3. L'algorithme AESGCM genere le ciphertext accompagne d'un **tag d'authentification** de 16 octets.
4. Le resultat est concatene : `[NONCE 12 octets][CIPHERTEXT + TAG]`, puis encode en Base64 URL-safe pour stockage en base.

```
def encrypt_health_data(data: str) -> str:
    nonce = os.urandom(12)
    ciphertext = aesgcm.encrypt(nonce, data.encode("utf-8"), None)
    return base64.urlsafe_b64encode(nonce + ciphertext).decode("utf-8")
```

3.3 En-tetes HTTP de securite (OWASP)

Un middleware personnalise injecte les en-tetes de securite sur chaque reponse :

En-tete	Valeur	Protection
Content-Security-Policy	default-src 'self'; script-src 'self' ...	Empeche les injections XSS
X-Frame-Options	DENY	Empeche le clickjacking
X-Content-Type-Options	nosniff	Empeche le MIME-sniffing
X-XSS-Protection	1; mode=block	Active le filtre XSS du navigateur
Referrer-Policy	strict-origin-when-cross-origin	Limite les fuites de referrer

3.4 Rate Limiting (SlowAPI)

L'endpoint `/predict` est limite a **5 requetes par minute** par IP via le decorateur `@limiter.limit("5/minute")`. Ce mecanisme protege contre les abus et les attaques par deni de service.

3.5 CORS (Cross-Origin Resource Sharing)

Le middleware CORS est configure pour n'autoriser que les origines de confiance, definies dans les variables d'environnement. Les methodes autorisees sont restreintes a `GET`, `POST` et `OPTIONS`.

4. Tests unitaires et d'integration (C8)

La strategie de test couvre deux axes : les tests fonctionnels de l'API et les tests de securite du module de chiffrement.

4.1 Tests de l'endpoint de prediction

Sept tests couvrent l'endpoint `/predict` :

Test	Description	Assertion
test_predict_unauthorized	Requete sans cle API	HTTP 403
test_predict_success	Requete valide avec cle API	HTTP 200 + champ predicted_weight_delta_grams
test_predict_validation_error_bmr	BMR hors bornes (< 500)	HTTP 422
test_predict_validation_error_calories	Calories hors bornes (> 900)	HTTP 422
test_predict_validation_error_portion	Portion hors bornes (< 1)	HTTP 422
test_predict_internal_server_error	Modele absent (mock)	HTTP 500
test_predict_rate_limiting	Depassement du rate limit	HTTP 429

4.2 Tests du module de securite

Deux tests verifient le comportement du chiffrement AES-256 :

- **test_encryption_decryption_integrity** : verifie que le cycle chiffrement/dechiffrement conserve l'integralite des donnees.
- **test_encryption_output_is_different_for_same_input** : verifie que le nonce aleatoire produit des chiffres differents pour un meme texte clair (protection contre l'analyse de frequence).

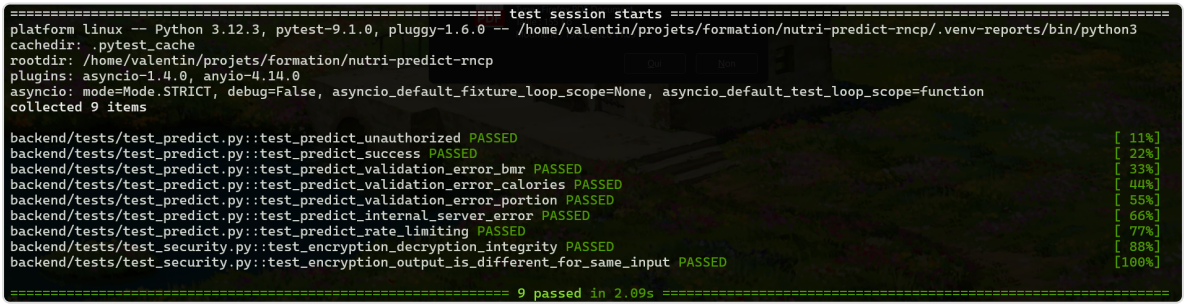


Figure 4 - Resultats Pytest : 9 tests passes (2.09 secondes)

5. Interface frontend

L'interface utilisateur est developpee en HTML/CSS/JavaScript pur (Vanilla JS), sans framework. Elle offre deux fonctionnalites principales :

- **Recherche de produit** : saisie d'un code-barres OpenFoodFacts pour recuperer les informations nutritionnelles via l'API.
- **Simulation IA** : saisie des parametres physiologiques (BMR, portion, calories) pour obtenir la prediction d'impact sur le poids.

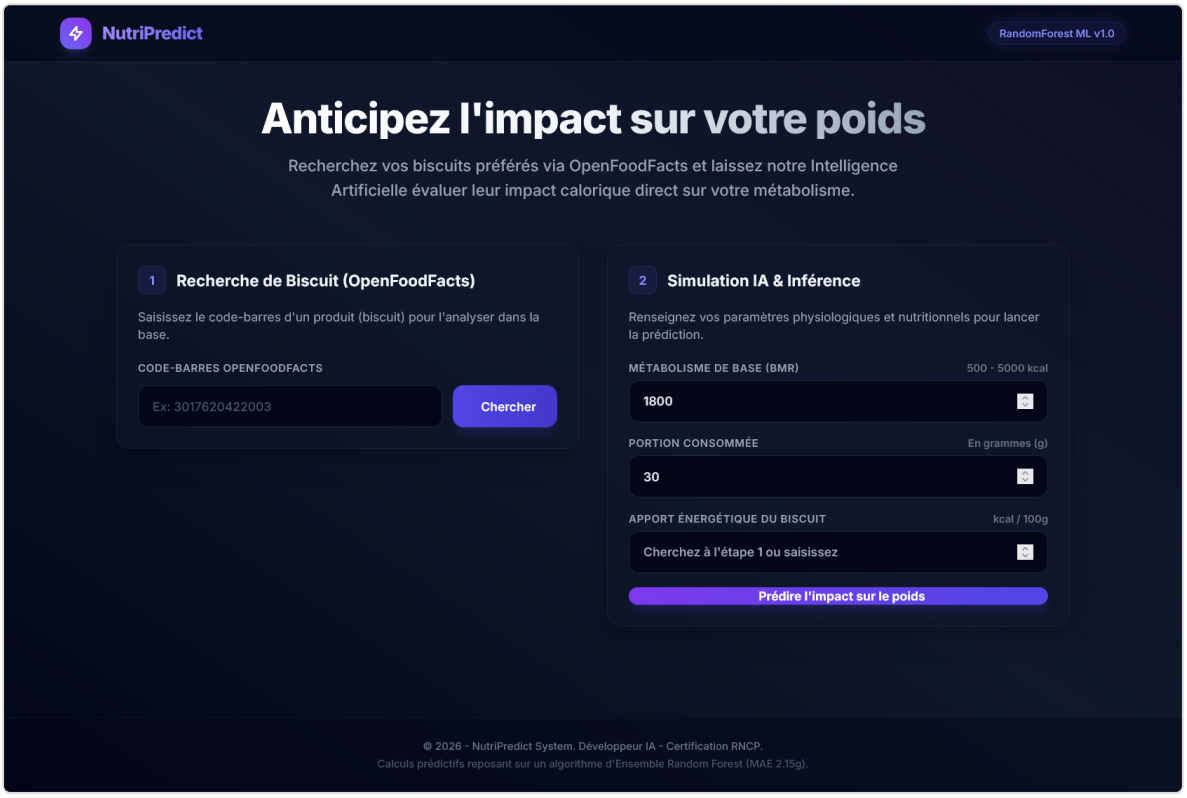


Figure 5 - Interface NutriPredict : recherche produit et simulation IA



Figure 6 - Resultat de prediction : deficit de -26.50 grammes

6. Conteneurisation Docker

L'application est conteneurisee via un `Dockerfile` multi-etapes base sur `python:3.12-slim`. Le fichier `docker-compose.yml` orchestre 4 services :

Service	Image	Port	Role
db	postgres:16-alpine	5432	Base de donnees relationnelle
api	nutri-predict-rncp-api (build)	8000	API FastAPI
prometheus	prom/prometheus:latest	9090	Collecte de metriques

grafana	grafana/grafana:latest	3000	Visualisation des metriques

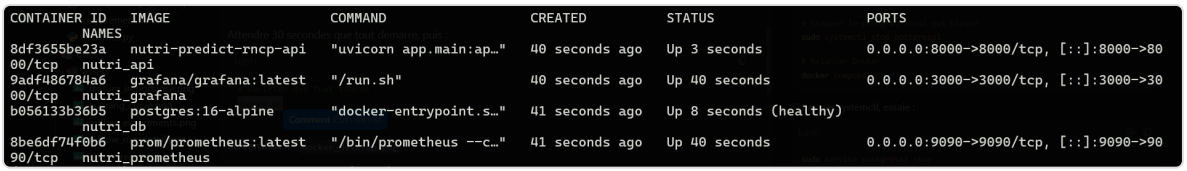


Figure 7 - Docker : 4 conteneurs en execution